

CONFIDENTIAL

VULNERABILITY ASSESSMENT & PENETRATION TEST

# Security Assessment Report

## *GMR Sunrise — Transformation Maturity Dashboard*

|                     |                                                                                          |
|---------------------|------------------------------------------------------------------------------------------|
| Application         | GMR Sunrise — Transformation Maturity Dashboard                                          |
| Version tested      | 1.0.0                                                                                    |
| Application type    | Frontend-only single-page application (React SPA)                                        |
| Hosting (as tested) | Azure Static Web Apps (static files + CDN)                                               |
| Live URL tested     | <fill in the deployed Azure URL>                                                         |
| Assessment type     | Grey-box — static code & config review, dependency audit, and automated dynamic scanning |
| Prepared by         | <your name / company>                                                                    |
| Prepared for        | <customer name>                                                                          |
| Assessment date     | <start> – <end>                                                                          |
| Report version      | 1.0 — <date>                                                                             |

01 Executive summary

A security assessment was performed on the GMR Sunrise Transformation Maturity Dashboard prior to its deployment into the customer's Microsoft 365 / Azure environment. The assessment combined a **manual review of the complete application source code and configuration**, a **software-composition (dependency) audit**, and **automated dynamic scanning** of the deployed application using a commercial VAPT platform.

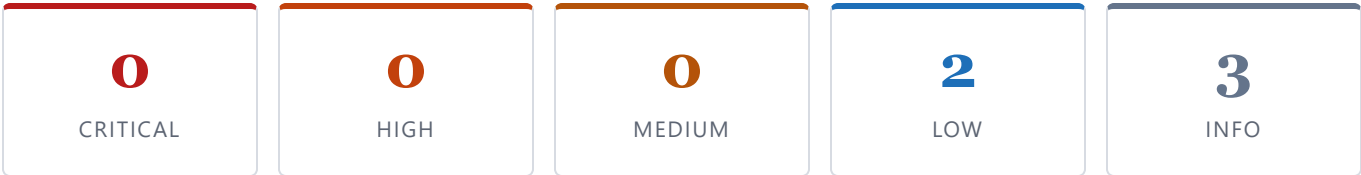
The application is a **frontend-only single-page application** with **no backend server, no application database, and no server-side code**. All logic executes in the end-user's browser. It is **strictly read-only** — it renders a single Excel workbook from SharePoint and provides **no facility to edit or write data** of any kind. It authenticates users against **Microsoft Entra ID** using the **OAuth 2.0 Authorization Code flow with PKCE**, and reads the workbook through the **Microsoft Graph API** using **delegated, read-only** permissions. Because there is no server-side attack surface and no write path, entire vulnerability classes — SQL injection, server-side request forgery, server RCE, insecure direct object references, data tampering — are **not applicable**.

**Scope of this rating.** This assessment reflects the application as deployed on **Azure Static Web Apps**. If it is later migrated to a different hosting platform, the static/code findings remain valid, but the dynamic checks (§8) should be re-run against the new URL and the platform-portability item **F-05** must be actioned.

OVERALL  
RISK  
RATING  
**LOW**

No critical, high, or medium-severity vulnerabilities were identified. The application demonstrates a strong security posture. A small number of low-severity and informational observations were recorded, each a well-understood trade-off for a single-page application, and each with a recommendation.

Findings at a glance



## 02 Scope

---

### In scope

| TARGET                            | DETAIL                                      |
|-----------------------------------|---------------------------------------------|
| Deployed web application          | <live URL> (Azure Static Web Apps)          |
| Application source code           | Complete React/Vite SPA source (18 files)   |
| Hosting / header configuration    | <code>staticwebapp.config.json</code>       |
| Authentication & session handling | Microsoft Entra ID + MSAL (PKCE)            |
| Authorization model               | Viewer vs Editor behaviour                  |
| Client → Graph data flow          | Token handling, scope usage, data rendering |
| Dependencies (supply chain)       | All npm packages shipped in the build       |

### Out of scope

Third-party services operated by Microsoft and Google were **not** tested — they are outside the customer's authority to authorize and are covered by their providers' own assurance programmes: Microsoft Entra ID ( `login.microsoftonline.com` ), Microsoft Graph ( `graph.microsoft.com` ), SharePoint Online ( `*.sharepoint.com` ), the Azure Static Web Apps platform, and Google Fonts. The assessment covered **how the application uses** these services, not the services themselves.

### 03 Methodology

The engagement followed industry-standard methodologies: **OWASP Top 10 (2021)**, the **OWASP Web Security Testing Guide (WSTG)**, and the **OWASP Application Security Verification Standard (ASVS)**. Techniques applied:

| TECHNIQUE           | DESCRIPTION                                                                                                           |
|---------------------|-----------------------------------------------------------------------------------------------------------------------|
| SAST                | Manual review of all source code and build/hosting configuration.                                                     |
| SCA                 | Dependency vulnerability audit against the full dependency tree.                                                      |
| DAST                | Automated scanning of the deployed app with a commercial VAPT platform (crawl, active checks, TLS/header inspection). |
| Manual verification | Authentication flow, authorization boundaries, client-side storage, header enforcement.                               |

**Methodology transparency.** Findings marked `[Static]` were confirmed by source/configuration review. Items marked `[Dynamic]` are validated by the automated VAPT platform against the live URL — see §8 for the checklist and where to record results.

#### Risk rating

Severity is rated using **CVSS v3.1**: Critical 9.0–10.0, High 7.0–8.9, Medium 4.0–6.9, Low 0.1–3.9, Informational 0.0 (no direct exploitability).

## 04 Application architecture

---

### End-user's web browser

React SPA — static files served from Azure Static Web Apps. All logic runs here.

▼ 1. OAuth 2.0 Authorization Code + PKCE ▲ ID + access token

### Microsoft Entra ID

OUT OF SCOPE

Sign-in & token issuance (Azure AD).

▼ 2. Bearer access token ▲ Excel workbook (.xlsx)

### Microsoft Graph → SharePoint Online

OUT OF SCOPE

Delegated, read-only access to one workbook.

*No vendor-controlled server · no database · no third-party analytics · tokens held in sessionStorage (cleared on tab close) · read-only.*

**Security-relevant properties:** no vendor server (no server-side surface); all logic client-side (surface = XSS, session/token handling, CSP, dependencies, OAuth config); read-only (no write path to SharePoint); client/tenant IDs are public identifiers, not secrets — no client secret, password, or API key exists in the application.

05 Findings summary

| ID   | FINDING                                                                            | SEVERITY | CVSS | STATUS             |
|------|------------------------------------------------------------------------------------|----------|------|--------------------|
| F-01 | Content-Security-Policy permits <code>'unsafe-inline'</code> for styles            | LOW      | 3.1  | Open (accepted)    |
| F-02 | Access tokens accessible to same-origin JavaScript ( <code>sessionStorage</code> ) | LOW      | 3.1  | Open (mitigated)   |
| F-03 | Public configuration values embedded in the JS bundle                              | INFO     | 0.0  | By design          |
| F-04 | Third-party resource load (Google Fonts)                                           | INFO     | 0.0  | Open (optional)    |
| F-05 | Security headers/CSP are hosting-platform-specific (Azure SWA)                     | INFO     | 0.0  | Open (portability) |

F-01 CSP permits 'unsafe-inline' for styles

LOW STATIC

CVSS v3.1 3.1 — AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N  
OWASP A05:2021 – Security Misconfiguration  
Component staticwebapp.config.json → Content-Security-Policy

DESCRIPTION

The CSP is strict for scripts (script-src 'self', object-src 'none', frame-ancestors 'none'), but style-src includes 'unsafe-inline' to support the styling approach (Tailwind utility classes, Framer Motion inline styles):

```
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
```

IMPACT

'unsafe-inline' for styles permits injected inline CSS, theoretically abusable for limited UI-redress or CSS-based inference. **Script injection remains fully blocked**, so practical risk is low — an attacker cannot execute JavaScript via this weakness.

RECOMMENDATION

Accept as a documented trade-off, or (if policy prohibits 'unsafe-inline') migrate to nonce- or hash-based styles. No functional change is required for safe operation.

## F-02 Access tokens accessible to same-origin JavaScript

LOW

STATIC

CVSS v3.1 3.1 — AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N

OWASP A02:2021 / A07:2021

Component `src/authConfig.js` → `cacheLocation: "sessionStorage"`

### DESCRIPTION

MSAL caches tokens in `sessionStorage`, which is readable by any JavaScript on the app's origin. There is no more secure alternative for a pure SPA without a backend (an `HttpOnly` cookie would require a server the app deliberately does not have).

### IMPACT

A successful XSS attack could read a token and impersonate the user against Graph for the token's lifetime. **This is contingent on an XSS vulnerability existing**, which the assessment did not find.

### MITIGATING CONTROLS PRESENT

- `sessionStorage` (not `localStorage`) — cleared when the tab closes.
- Strict CSP `script-src 'self'`.
- No `eval`, `dangerouslySetInnerHTML`, or `innerHTML` in the source.
- Delegated, read-only Graph scopes — a stolen token cannot write data.

### RECOMMENDATION

No change required; this is the recommended pattern for a backend-less SPA. Maintain the strict CSP and safe-rendering practices that keep XSS out.



### F-03 Public configuration values embedded in the JS bundle

INFO

STATIC

CVSS v3.1 0.0

Component Build output; `.env.example`, `src/authConfig.js`

#### DESCRIPTION

`VITE_CLIENT_ID` and `VITE_TENANT_ID` are compiled into the JS bundle at build time.

#### IMPACT

**None.** These are **public identifiers** present in the client bundle of every SPA. They are not secrets and cannot authenticate without a valid user credential and the PKCE flow. The assessment confirmed no client secret, password, API key, connection string, or token anywhere in the source or bundle.

#### RECOMMENDATION

No change required. `.env.example` documents this correctly and `.env` is git-ignored.

### F-04 Third-party resource load (Google Fonts)

INFO

STATIC / DYNAMIC

CVSS v3.1 0.0

Component `index.html` font links; CSP `font-src` / `style-src`

#### DESCRIPTION

The app loads web fonts from Google (`fonts.googleapis.com`, `fonts.gstatic.com`), both allow-listed in the CSP. End-user browsers therefore make requests to Google for typography.

#### IMPACT

A minor **privacy** consideration (Google receives request metadata such as IP address), not a security vulnerability. No application or programme data is sent.

#### RECOMMENDATION

Acceptable for most organisations. If the customer's privacy policy disallows third-party font hosting, self-host the fonts and remove the two Google hosts from the CSP.

## F-05 Security headers/CSP are hosting-platform-specific (Azure SWA)

INFO

STATIC · PORTABILITY

CVSS v3.1 0.0 — deployment portability

Component `staticwebapp.config.json`

### DESCRIPTION

The application's entire HTTP-header defence — CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, COOP — is delivered through `staticwebapp.config.json`, which is **only honoured by Azure Static Web Apps**. On any other host this file is **silently ignored** and none of these protections apply unless re-implemented using that platform's own mechanism.

### IMPACT

No impact while hosted on Azure Static Web Apps (as tested). If migrated to another platform without re-implementing the headers, the app would lose its CSP, HSTS, clickjacking, and MIME-sniffing protections — a significant security regression.

### RECOMMENDATION

If hosting is moved, re-create the same headers using the target platform's mechanism, then re-verify (§8, D-1). Equivalents:

| PLATFORM                           | HEADER MECHANISM                                                                   |
|------------------------------------|------------------------------------------------------------------------------------|
| Azure Static Web Apps<br>(current) | <code>staticwebapp.config.json</code> — in place                                   |
| Vercel                             | <code>vercel.json</code> → <code>headers</code>                                    |
| Netlify                            | <code>netlify.toml</code> <code>[[headers]]</code> or a <code>_headers</code> file |
| AWS S3 + CloudFront                | CloudFront <b>Response Headers Policy</b>                                          |
| Azure Blob static website          | Azure <b>Front Door</b> / CDN rules (Blob alone cannot set headers)                |
| Cloudflare Pages                   | <code>_headers</code> file / Transform Rules                                       |
| GitHub Pages                       | <b>Not supported</b> — cannot set HTTP headers. Not recommended for this app.      |
| nginx / Apache                     | <code>add_header</code> / <code>Header set</code> in server config                 |

After any migration, also re-configure the SPA fallback route (deep links → `index.html`) and update the Entra redirect URI to the new domain.

## 07 Positive security observations

---

- ✓ **Strictly read-only.** No edit or write capability in the UI and no write path to SharePoint — the dashboard only visualises data.
- ✓ **No dangerous DOM sinks.** No `eval`, `dangerouslySetInnerHTML`, `innerHTML`, or `document.write` anywhere in the source.
- ✓ **Safe rendering.** All external (workbook) data is rendered as auto-escaped text by React.
- ✓ **Zero known dependency vulnerabilities.** `npm audit` — 0 across 138 dependencies.
- ✓ **Strict CSP for scripts.** `script-src 'self'`, `object-src 'none'`, `frame-ancestors 'none'`, `base-uri 'self'`.
- ✓ **Full HTTP security-header set.** HSTS, X-Frame-Options: DENY, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, COOP.
- ✓ **Modern authentication.** OAuth 2.0 Authorization Code + PKCE, no client secret.
- ✓ **Least-privilege data access.** Delegated, read-only Graph scopes: `User.Read`, `Files.Read.All`, `Sites.Read.All`.
- ✓ **Reduced token exposure.** Tokens in `sessionStorage` (cleared on tab close), not `localStorage`.
- ✓ **No reverse-tabnabbing.** External link uses `rel="noreferrer"`.
- ✓ **HTTPS enforced.** HSTS + `upgrade-insecure-requests`.

## 08 Dynamic checks to complete with the VAPT platform

Run the following against the **live URL** with your commercial VAPT platform and record the outcomes. These validate, at runtime, controls confirmed statically above.

| #    | DYNAMIC CHECK                                   | EXPECTED RESULT                         | TOOL RESULT |
|------|-------------------------------------------------|-----------------------------------------|-------------|
| D-1  | Security-header scan                            | A / A+; all headers present             | <paste>     |
| D-2  | TLS/SSL configuration scan                      | TLS 1.2+; no weak ciphers; valid cert   | <paste>     |
| D-3  | Authenticated crawl + active XSS/injection scan | No script-executing findings            | <paste>     |
| D-4  | CSP effectiveness / bypass attempt              | Inline/external script blocked          | <paste>     |
| D-5  | Clickjacking (framing) test                     | Page refuses to frame (DENY)            | <paste>     |
| D-6  | Auth: unassigned account                        | Blocked by Entra before app loads       | <paste>     |
| D-7  | Authz: Viewer vs Editor                         | No data change possible either way      | <paste>     |
| D-8  | Session: sign-out & token expiry                | Tokens cleared; no reuse after sign-out | <paste>     |
| D-9  | Sensitive-data-in-response scan                 | No secrets/tokens/PII in served assets  | <paste>     |
| D-10 | OAuth redirect-URI / state handling             | No open-redirect; state/PKCE enforced   | <paste>     |

Attach the platform's raw report as **Appendix B**. If any dynamic check produces a finding, add it to §5/§6 with a CVSS rating and update the counts on the cover.

## 09 Conclusion

---

Based on the static assessment completed and pending confirmation of the dynamic checks in §8, the GMR Sunrise Transformation Maturity Dashboard presents a **LOW overall security risk** and is considered **suitable for deployment**. No critical, high, or medium-severity vulnerabilities were identified. The residual low/informational items are documented, understood trade-offs appropriate to a strictly read-only, backend-less single-page application.

### Recommended actions before go-live

1. Complete the §8 dynamic checks with the VAPT platform and attach the output.
2. Perform the post-deployment CSP verification (confirm the SharePoint download host is allow-listed — see the handover document).
3. Optionally address F-01 / F-04 if required by the customer's policies.
4. If hosting is ever moved off Azure Static Web Apps, action **F-05** (re-implement the security headers) and re-run §8 against the new URL.

## 10 Appendices

---

**Appendix A** Tooling — manual source review; `npm audit` (SCA); *<commercial VAPT platform name / version>* (DAST).

**Appendix B** Raw automated scan report (attach export).

**Appendix C** References — OWASP Top 10 (2021), OWASP WSTG, OWASP ASVS, CVSS v3.1 specification.

---

*This report reflects the security posture of version 1.0.0 at the assessment date. Any subsequent code, dependency, or configuration change should be re-assessed. Ratings for statically-confirmed findings are the assessor's; dynamic findings are subject to the VAPT platform's output.*